

Cosa significa "ottimizzare" un codice?



renderlo più efficiente
migliorare un parametro

ridurre tempi esecuzione
ridurre la memoria richie
ottenere efficienza massi
rimuovere inefficienze
migliorare spazio e tempo
ottenere le massime prest
massimizzare l'efficienza
abbassare la complessità
await and promise
migliorarlo
complessità minore
modularlo
resource managment
complessità più bassa
renderlo flessibile
max result - min effort
minimizzare il dispendio
ridurre i tempi
ridurre la complessità
ridurre numero operazioni

Come si misura il tempo di esecuzione di una porzione di codice?

5



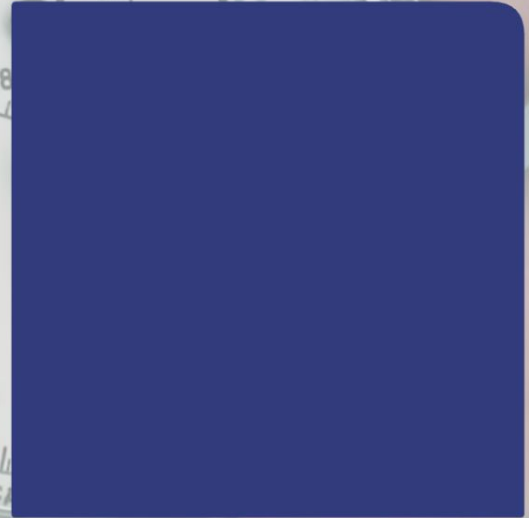
Con il comando time

2



Con il comando perf

11



Con la funzione clock_gettime

Cosa è il memory aliasing?

Boh

Boh

Ho sentito il termine ma non ricordo cosa sia di preciso

stessa risorsa ma con nome diverso

i don't know

Porzione di memoria doppia

Porzioni di memoria uguali

Dare più nomi ad una stessa risorsa in memoria



The screenshot shows a code editor with a dark theme. The file explorer on the left shows 'index.ts', '.env.local', and '_app.tsx'. The main editor displays the content of '_app.tsx'. The code is as follows:

```
1 import { useEffect } from 'react';
2 import Head from 'next/head';
3 import type { AppProps } from 'next/app';
4 import { ApolloProvider } from '@apollo/client';
5 import { ThemeProvider } from '@material-ui/core';
6 import CssBaseline from '@material-ui/core';
7 import { Container } from '@material-ui/core';
8 import { useApollo } from '../graphql/hooks';
9
10 import { lightTheme, darkTheme } from '../themes';
11 import useLocalStorage from '../hooks/useLocalStorage';
12
13 import NavBar from '../components/NavBar';
14
15 function App({ Component, pageProps }) {
16   const [currentTheme, setCurrentTheme] = useLocalStorage('theme', 'light');
17   const apolloClient = useApollo();
18
19   useEffect(() => {
20     const jssStyles = document.querySelector('#jss');
21     if (jssStyles) {
22       jssStyles.parentElement.removeChild(jssStyles);
23     }
24   }, []);
25
26   return (
27     <>
28     <Head>
29     <title>ECU-DEV</title>
30     <meta name="viewport" content="width=device-width, initial-scale=1" />
31     </Head>
32     <ThemeProvider theme={currentTheme}>
33     <CssBaseline>
34     <Container>
```

Cosa è il memory aliasing?

Sdoppiamento di porzioni di memoria

Memoria utilizzata da più processi

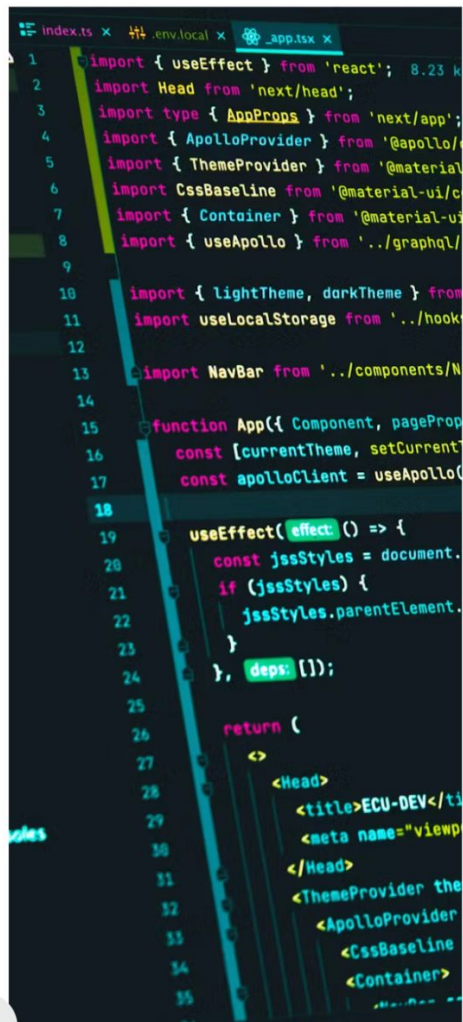
porzione di memoria che ha più nomi per maggiore chiarezza del codice

un alias è una copia di un'identità o simile, quindi in qualche modo questo concetto ma riguardante la memoria (?)

Indicare la stessa area di memoria con più nomi

riferimento ad una determinata porzione di memoria

nome diverso, stessa risorsa



```
1 import { useEffect } from 'react';
2 import Head from 'next/head';
3 import type { AppProps } from 'next/app';
4 import { ApolloProvider } from '@apollo/client';
5 import { ThemeProvider } from '@material-ui/core';
6 import CssBaseline from '@material-ui/core';
7 import { Container } from '@material-ui/core';
8 import { useApollo } from '../graphql';
9
10 import { lightTheme, darkTheme } from '../themes';
11 import useLocalStorage from '../hooks/useLocalStorage';
12
13 import NavBar from '../components/NavBar';
14
15 function App({ Component, pageProps }) {
16   const [currentTheme, setCurrentTheme] = useLocalStorage('theme', 'light');
17   const apolloClient = useApollo();
18
19   useEffect(() => {
20     const jssStyles = document.createElement('style');
21     if (jssStyles) {
22       jssStyles.parentElement.appendChild(jssStyles);
23     }
24   }, [currentTheme]);
25
26   return (
27     <>
28       <Head>
29         <title>ECU-DEV</title>
30         <meta name="viewport" content="width=device-width, initial-scale=1" />
31       </Head>
32       <ThemeProvider theme={currentTheme}>
33         <ApolloProvider client={apolloClient}>
34           <CssBaseline>
35             <Container>
```




Ottimizzazione 0 - *Misurare le Performance*

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania



INAF
ISTITUTO NAZIONALE
DI ASTROFISICA



Misurare le performance



Espressione delle performance

In queste slide introduciamo una metrica per stimare la performance di un codice nello sfruttare alcune delle risorse della CPU.

In particolare, ci concentreremo su

- (i) quanto è "veloce" un codice e
- (ii) quanto bene un codice sfrutta la capacità di parallelismo a livello di istruzione (instruction-level parallelism) di una CPU.

Le metriche che introdurremo sono più adatte per quelle sezioni di codice che eseguono cicli (loop), le quali rappresentano in effetti una frazione ampia e significativa dei codici scientifici in generale.



Il Ciclo di Clock

Come sapete, una CPU non lavora "continuamente". Al contrario, la sua attività è regolata da un clock interno il cui ritmo è dell'ordine di miliardi di battiti al secondo. La frequenza tipica della CPU è tra 2 e 4 GHz, il che significa che il tempo impiegato da un "ciclo di clock" è dell'ordine di 0,5 - 0,35 ns.

Ci riferiremo a questo intervallo di tempo come "un ciclo". Inoltre, ai fini del risparmio energetico, le CPU hanno capacità di throttling, il che significa che la frequenza del clock non è fissa ma può essere adattata al carico di lavoro. Maggiore è il carico di lavoro, più alto è il clock e viceversa.



Cicli per Elemento - CPE

A causa della variabilità del clock della CPU, misurare il "tempo-soluzione di wall-clock" non è sempre la metrica migliore, o almeno l'unica, da raccogliere allo scopo di valutare come si comporta effettivamente un frammento di codice.

Quantificare il numero di cicli di clock spesi su una sezione di codice può essere ancora più informativo che sapere quanti secondi ha richiesto per l'esecuzione.

Concentrarsi su sezioni di codice che ripetono un blocco di istruzioni su un array, che sono molto comuni e spesso rappresentano gli hotspot più intensivi dal punto di vista computazionale, si traduce facilmente in una metrica di cicli-per-elemento (Cycles-Per-Element, CPE).



Istruzioni per Ciclo - IPC

Infatti, saremmo interessati a sapere quanto è efficiente il nostro codice per-elemento piuttosto che per-iterazione poiché la nostra implementazione potrebbe essere in grado di elaborare più elementi per iterazione.

← Architetture particolari

Al contrario, nel caso volessimo stimare quanto bene stiamo sfruttando la capacità super-scalare della CPU, la metrica adeguata da considerare sarebbe la valutazione di quante istruzioni-per-ciclo (Instructions-Per-Cycle, IPC) vengono eseguite.

Vedremo nelle prossime lezioni come raccogliere queste metriche sofisticate nella pratica. Per ora, l'obiettivo è chiarire come la performance debba essere espressa e misurata.



Tempo di Esecuzione

Tuttavia, misurare il "tempo di esecuzione" di un codice è una metrica fondamentale che dovremmo raccogliere, almeno per una prima valutazione.

confronto: faranno il wall-clock

Fondamentalmente, si ha accesso a 3 diversi tipi di "tempo":

1. "wall-clock" time

Fondamentalmente lo stesso tempo che si può ottenere dall'orologio a muro in questa stanza. È una misura del "tempo assoluto". *→ semplice tempo da quando inizia a quando finisce*

Nei sistemi POSIX, è la quantità di secondi trascorsi dall'inizio dell'epoca Unix, ovvero l'1 gennaio 1970 00:00:00 UT.

2. "system-time"

La quantità di tempo che l'intero sistema ha trascorso eseguendo il vostro codice. Può includere I/O, chiamate di sistema, ecc.

→ tempo in kernel mode, accessi in memoria ecc..

3. "process user-time"

La quantità di tempo speso dalla CPU per eseguire le istruzioni del vostro codice, in senso stretto. *→ esclude i momenti "morti"*

→ tempo in user mode in cui si eseguono codici

in cui si aspetta



Tempo di Esecuzione

Tuttavia, misurare il "tempo di esecuzione" di un codice è una metrica fondamentale che dovremmo raccogliere, almeno per una prima valutazione.

Fondamentalmente, si ha a

1. "wall-clock" time

Fondamentalmente lo s
dall'orologio a muro in
assoluto.

Nei sistemi POSIX, è la
dell'epoca Unix, ovvero

2. "system-time"

La quantità di tempo d
vostro codice. Può incl

3. "process user-time"

La quantità di tempo s
vostro codice, in senso

Sistemi POSIX

POSIX è un insieme di standard IEEE
intesi a definire un ambiente standard e
un'API standard per le applicazioni,
nonché il comportamento atteso delle
applicazioni. Estende l'API C, ad
esempio, le utility CLI, il linguaggio shell
e molte altre cose. Tutti i sistemi *NiX
sono sistemi POSIX. Altri sistemi
conformi sono:

- AIX (IBM)
- OSX (Apple)
- HP-UX (HP)
- Solaris (Oracle)



Misurare i tempi

È possibile misurare tutti i tempi citati:

Al di fuori del codice

- Misurate l'intera esecuzione del codice
- Chiedete al sistema operativo (OS) di misurare il tempo impiegato dal vostro codice per l'esecuzione:
 - Usando il comando `time` (vedi `man time`)
 - Usando il profiler `perf`
 - ... scoprite altri modi sul vostro sistema

All'interno del vostro codice

- Potete misurare sezioni separate del codice
- Accedete alle funzioni di sistema per accedere al contatore di sistema

- ❑ Quale tempo ci serve? (Real, User, System, ...)
- ❑ Di quale precisione abbiamo bisogno? (1s, 1ms, 1us, 1ns)
- ❑ Di quale tempo di *wrap-around* abbiamo bisogno?
- ❑ Abbiamo bisogno di un clock monotono?
- ❑ Abbiamo bisogno di una chiamata di funzione portabile?



Misurare i tempi

Principio di base:
chiamate la
funzione di sistema
corretta subito
prima e subito dopo
il frammento di
codice che vi
interessa e calcolate
la differenza (sì,
state includendo
l'overhead della
funzione di tempo).

`gettimeofday(...)` restituisce il tempo *wall-clock* con precisione di μs .

I dati sono forniti in una struttura `timeval`:

→ 6 prendi dall'inizio e
alla fine

```
struct timeval {  
    time_t      tv_sec;    // secondi  
    useconds_t  tv_usec;   // microsecondi  
};
```

`clock_t clock()` restituisce *user-time + system-time* con precisione di μs . I

risultati devono essere divisi per `CLOCKS_PER_SEC`. → ignora il tempo di pausa

`int clock_gettime(clockid_t clk_id, struct timespec ...):`

- `CLOCK_REAL_TIME`: clock in tempo reale a livello di sistema; → wall-clock
 - `CLOCK_MONOTONIC`: tempo monotono; → tempo trascorso da un momento preciso
 - `CLOCK_PROCESS_CPUTIME`: tempo CPU ad alta risoluzione per processo;
 - `CLOCK_THREAD_CPUTIME_ID`: tempo ad alta precisione per thread; ↗
 - La risoluzione è di 1 ns.
- user time + kernel time

```
struct timespec {  
    time_t      tv_sec;    // secondi  
    long        tv_nsec;   // nanosecondi};
```



Misurare i tempi

Principio di base:
chiamate la
funzione di sistema
corretta subito
prima e subito dopo
il frammento di
codice che vi
interessa e calcolate
la differenza (sì,
state includendo
l'overhead della
funzione di tempo).

```
int getrusage(int who, struct rusage *usage):
```

- `RUSAGE_SELF`: processo + tutti i thread
- `RUSAGE_CHILDREN`: l'intera gerarchia dei figli
- `RUSAGE_THREAD`: thread chiamante

```
struct rusage {  
    struct timeval ru_utime; /* user CPU time used */  
    struct timeval ru_stime; /* system CPU time used */  
    long ru_maxrss; /* maximum resident set size */  
    long ru_ixrss; /* integral shared memory size */  
    long ru_idrss; /* integral unshared data size */  
    long ru_isrss; /* integral unshared stack size */  
    long ru_minflt; /* page reclaims (soft page faults) */  
    long ru_majflt; /* page faults (hard page faults) */  
    long ru_nswap; /* swaps */  
    long ru_inblock; /* block input operations */  
    long ru_oublock; /* block output operations */  
    long ru_msgsnd; /* IPC messages sent */  
    long ru_msrgrcv; /* IPC messages received */  
    long ru_nsignals; /* signals received */  
    long ru_nvcsw; /* voluntary context switches */  
    long ru_nivcsw; /* involuntary context switches */  
};
```




Misurare i tempi

Una possibilità su un sistema POSIX è:

```
#define CPU_TIME (clock_gettime( CLOCK_PROCESS_CPUTIME_ID, &ts ), \
                    (double)ts.tv_sec + \
                    (double)ts.tv_nsec * 1e-9)
```

....

```
Tstart = CPU_TIME ;  
// your code segment here
```

```
Time = CPU_TIME - Tstart;
```



Accumulare dati sulle performance

Indipendentemente dalle metriche che state accumulando, affrontare solo 1 misura non è una buona stima: i sistemi informatici sono davvero complicati e molte cose accadono continuamente, soprattutto sulle architetture moderne, e potreste osservare variazioni significative nelle vostre metriche da un'esecuzione all'altra.

Pertanto, è necessario procedere "statisticamente", ovvero accumulando diverse misure e modellando l'overhead di misura e di sistema.

Ad esempio, acquisire il numero di cicli o il tempo richiede a sua volta un certo numero di cicli; un ciclo (loop) ha una quantità di overhead intrinseco. E così via. Molto spesso, fare la media su un numero "sufficiente" di esecuzioni e sottrarre l'overhead noto è sufficiente per ottenere una stima abbastanza buona.



Accumulare dati sulle performance

Esempi in pseudo-linguaggio:

```
double t_overhead;  
timing_overhead =  
get_time();  
for ( int i = 0; i < LOTS_OF_ITER;  
    i++ ) double this_time =  
    get_time();  
t_overhead = (get_time() -  
             t_overhead) /  
             LOTS_OF_ITER;
```

```
double timing = get_time();  
...  
block of code you want to characterize  
...
```

```
timing = get_time() - timing - t_overhead;
```

```
double timing = 0;  
double stddev;  
for( int i = 0; i < MANY_ITER; i++ )  
{  
    ...  
    double time0 = get_time();  
    block_to_be_measured  
    double time1 = get_time();  
    double elapsed = time1 - time0;  
    timing += elapsed - t_overhead;  
    stddev += elapsed * elapsed;  
    ...  
}  
  
timing = timing/MANY_ITER;  
stddev = sqrt( stddev/MANY_ITER - timing*timing);
```

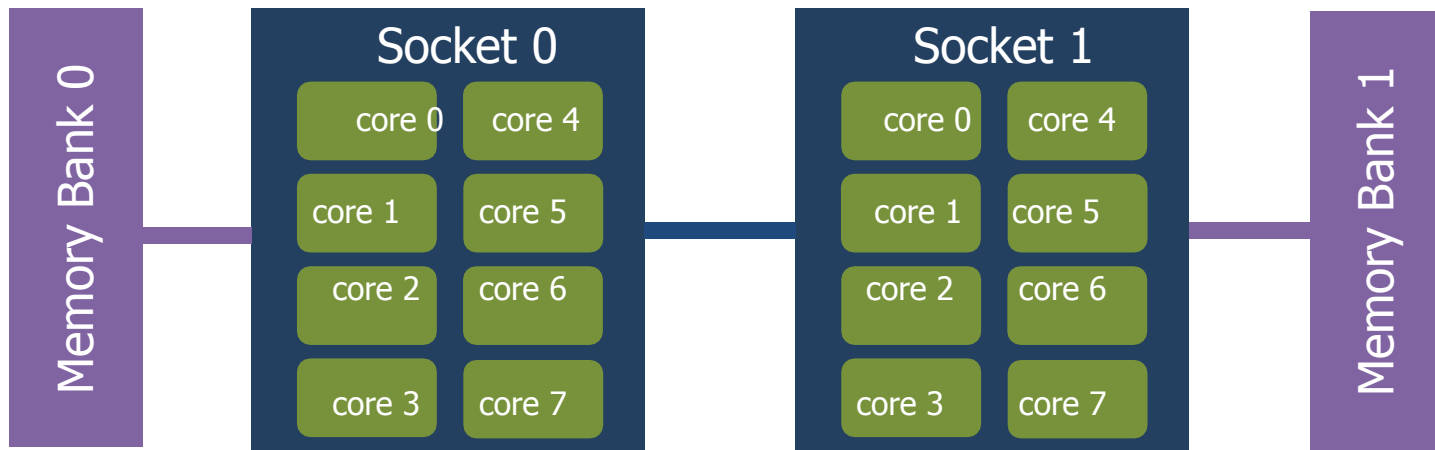
Handwritten annotations in blue:

- Arrow from `timing` to `double timing = 0;`: *summa su cui si fa la media*
- Arrow from `time0` to `double time0 = get_time();`: *inizio*
- Arrow from `time1` to `double time1 = get_time();`: *fine*
- Arrow from `elapsed` to `double elapsed = time1 - time0;`: *effettivo*
- Arrow from `timing` to `timing += elapsed - t_overhead;`: *esclusione get_time*
- Arrow from `stddev` to `stddev += elapsed * elapsed;`: *serie per la dev standard*



Accumulare dati sulle performance

All'interno di un socket ci sono molti core, da **4-8 nelle CPU commerciali** che si trovano su laptop o computer desktop fino a circa 64 nelle CPU di fascia alta per server e nodi di calcolo.



L'OS può migrare i thread del vostro codice da un core all'altro e anche da un socket all'altro. Se anche i dati vengono migrati dipende dalla politica adottata.



Accumulare dati sulle performance

Quindi, eseguire un programma senza associarlo a un core specifico e a un memory bank specifico può portare a un comportamento non ottimale. Perciò, quando siete interessati a profilare un codice, dovete essere sicuri che non venga migrato.

Potete richiederlo al sistema operativo utilizzando i comandi

```
taskset 0 numactl
```

Basta dare un'occhiata alle relative pagine `man` per tutti i dettagli.



Accumulare dati sulle performance

`numactl -H`

espone la topologia del nodo

`numactl`

lega l' esecuzione al core n

`--cpunodebind= $n$`

`numactl -C  $n$`

`numactl`

lega la memoria al banco di memoria associato al core n

`--membind= $n$  numactl`

`-m  $n$`

`numactl -C +list`

lega l' esecuzione ai core *relativi* elencati in *list*.

*Es.: numactl -C +0,2,4 prog.x eseguirà
prog.x sui core 0,2 e 4 del cpuset dato al job.*



Ottimizzazione 1 - *Introduzione e Compilatori*

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania



INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

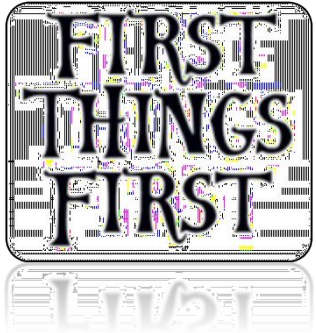
Obiettivi

Iniziamo una serie di lezioni sull'"ottimizzazione" del codice su single-core.

Questa lezione riguarda i concetti generali e le premesse sul significato di "ottimizzazione", mentre nelle lezioni successive approfondiremo i concetti e le caratteristiche più importanti.

La diapositiva successiva presenta lo schema generale delle lezioni sull'ottimizzazione.

Outline generale



Intro



Cache &
Memoria



Cicli



Branch

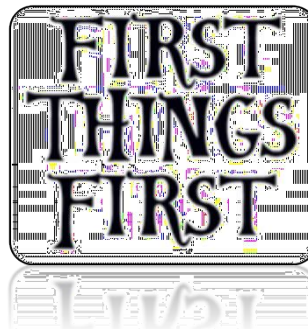


Pipeline

Outline di questa lezione



Introduzione &
concetti generali



Alcuni dettagli
preliminari
su compilatore & memoria



L'High Performance Computing richiede, come suggerisce il nome stesso, di ottenere la massima efficacia dal codice sulla macchina su cui viene eseguito.

↳ teniamo conto della
macchina su cui viene
eseguito

"Ottimizzare" è ovviamente un passaggio chiave in questo processo.

Tuttavia, sia "*ottimizzazione*" che "*efficacia*" devono essere definite.

Potrebbe non esistere un codice "ottimale" tout-court.

Provate a definire cosa sia "ottimale"...



Teniamo conto di tutte queste cose

VINCOLI DI MEMORIA

uso della memoria (dati & codice)

VICOLI ENERGETICI

VINCOLI DI I/O

progettazione accurata di I/O
(disco, memoria, rete,..)

VINCOLI DI TEMPO

time-to-solution
(caso peggiore? caso ottimale? "caso medio"?)

“ OTTIMIZZAZIONE ”

ROBUSTEZZA/RESILIENZA
mission-critical



Premature optimization is the root of all evil

D. Knuth

Ciò significa che, anche se alcune delle cose che imparerai possono sembrare interessanti, la tua prima attenzione deve essere rivolta a:

(i) la **correttezza** del codice,

(ii) il **modello di dati**,

(iii) l'**algoritmo** che scegli.

*prima si fa tutto in maniera
corretta poi si ottimizza*

Un codice meravigliosamente ottimizzato che adotta l'algoritmo sbagliato è un controsenso, perché un algoritmo migliore, anche se implementato male, batterà (quasi) sempre il codice precedente per un numero di casi sufficientemente ampio.



Meglio iniziare a pensare in termini di codice "migliorato"

- Non vuoi danneggiare il tuo codice che svolge onestamente il suo compito: un codice più veloce che fornisce risultati errati non è ottimale in alcun modo.
 - Molto probabilmente dovrai riutilizzare quel codice dopo un po' di tempo. E avrai bisogno che sia *leggibile, manutenibile e ben progettato*.
→ Principi della program. sostenibile: nomi + commenti + funz. + ecc.
 - Altri molto probabilmente dovranno leggere, comprendere e riutilizzare quel codice: *un codice pulito, comprensibile, non oscuro e non ridondante* potrebbe non essere "migliorato" in qualche modo, ma in realtà migliora la qualità della tua vita!
-



Primi passi da considerare



Avere un **codice pulito**



Ri-progettare la sua architettura
fondamentale



Migliorare il flusso di lavoro, l'impronta di
memoria, l'utilizzo delle risorse, il tempo di
risoluzione, ...



Dry-ness



Non aggiungere codice non necessario né duplicare codice.

Il codice **non necessario** aumenta la quantità di lavoro necessaria

- per **mantenere il codice**, sia per il debug che per l'aggiornamento
- per **estendere le sue funzionalità**.

*usa le
funzioni*

Il codice **duplicato** aumenta il debito tecnico, che ha già un numero sufficientemente elevato di fonti:

- stile di programmazione copia-e-incolla
- approccio troppo *agile*
- codifica rigida, correzioni rapide e sporche, cargo-cult, sciatteria

DRY:
Don't
Repeat
Yourself



Leggibilità



La **leggibilità** è un pilastro della **manutenibilità**.

Manutenibilità significa che il software può essere esteso, aggiornato, debuggato, corretto.

Base: scrivi commenti

Avanzato: SCRIVI COMMENTI ! (possibilmente, usa strumenti simili a *doxygen*)

X-avanzato: evita commenti stupidi, ovvi, inutili, confusi

Tieni presente: "il codice è la man page definitiva"



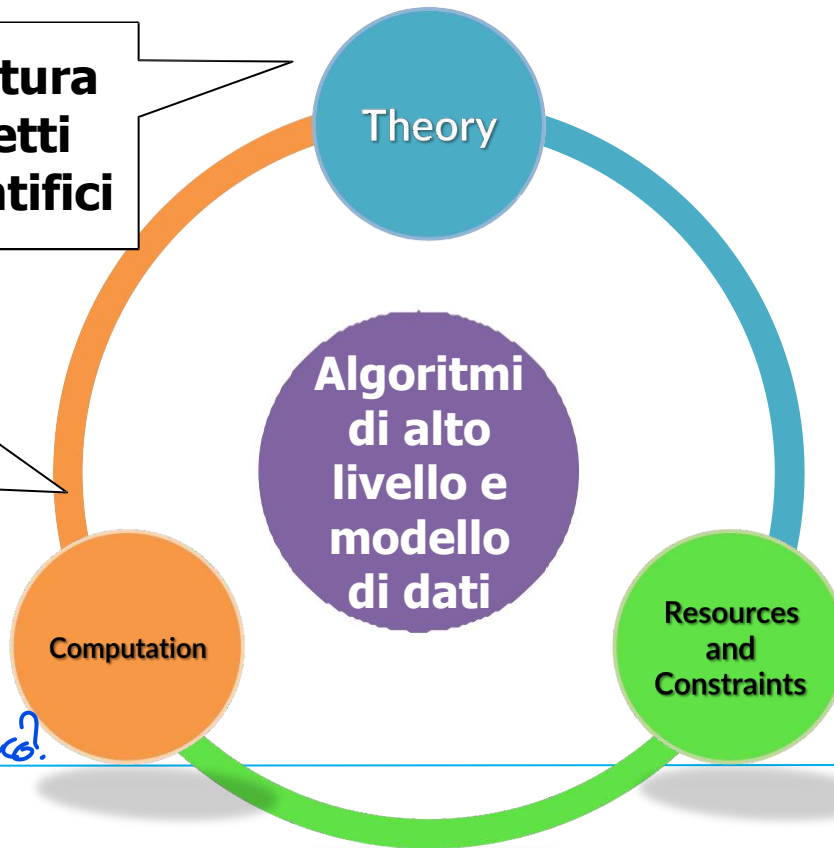
Design

Come si fa di solito?

- **Consulta la letteratura**
- **Comprendi gli aspetti matematici e scientifici**

Non sei alla lavagna, ma su un sistema digitale discreto. Le equazioni devono essere tradotte.

come lo traduci?



Concorrenza e parallelismo

- **Vincoli di memoria**
- **Vincoli di I/O**
- **Vincoli energetici**

che risorse ho a disposizione



Design



Immagine astratta più alta:

- Multi-componenti interagenti
- MODELLO DI DATI di alto livello

Sottosistemi e moduli
Strategie di
comunicazione

Moduli come
produttori di compiti
complessi



Dettagli di
implementazione

- molti algoritmi
- concorrenza

Struttura
logica
dei
moduli

interazioni

Codifica e
dettagli di
livello molto
basso



ABSTRACTION

CODING
DETAIL



II TESTING fa parte del DESIGN

- Unit test
 - stressare separatamente ogni unità del codice
- Integration test
 - stressare il comportamento integrato di tutte le unità insieme
- System test



VALIDAZIONE e VERIFICA fanno parte del design

- La **validazione** assicura che il codice faccia ciò che doveva fare → *fa quello che deve*
 - (tutti i moduli sono progettati in base alle specifiche di progettazione)
- La **verifica** assicura che il codice faccia ciò che fa correttamente → *fa bene*
 - (test black-box contro casi di test, ...)



Migliora il codice



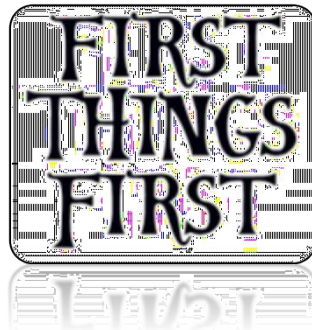
Migliorare il flusso di lavoro, l'impronta di memoria, l'utilizzo delle risorse, il tempo di soluzione, ...

Questo sarà il focus delle prossime lezioni!

Outline



Introduzione &
concetti generali



Alcuni dettagli
preliminari
su compilatore & memoria

First things first

“Ottimizzazione” può essere un'etichetta per diversi concetti, come abbiamo visto all'inizio di questa lezione.

Molti fatti e fattori concorrenti devono essere tenuti insieme, ed è piuttosto difficile dare affermazioni generali su quali siano i più fondamentali.

Il design di alto livello gioca un ruolo chiave, così come la scelta degli algoritmi e, infine, i dettagli di implementazione. A volte, ma solo a volte, anche una singola riga di codice – ad esempio un prefetching – può avere conseguenze brillanti (o disastrose) su una parte limitata del codice.

First things first

1. Il primo obiettivo è avere un programma che fornisca le **risposte corrette** e si comporti correttamente in tutte le condizioni. Il codice deve essere il più chiaro, pulito, conciso e documentato possibile.
 2. Il primo passo verso l'ottimizzazione è adottare gli **algoritmi** e le strutture dati **più adatti**. Cosa sia "più adatto" deve essere correlato alla convoluzione dei vincoli che inquadrano la tua attività (tempo di soluzione, energia di soluzione, memoria, ...).
Il secondo passo è che il **codice sorgente** sia **ottimizzabile dal compilatore**. Quindi, devi avere una solida comprensione delle capacità e dei limiti dei **compilatori, così come per la tua architettura target**. Comprendi il miglior compromesso tra portabilità e "prestazioni" (tieni conto dello sforzo umano in quest'ultimo).
 3. Il terzo passo è ottenere un **flusso di lavoro basato sui dati e sui compiti** che, possibilmente, quasi certamente, sarà parallelo in paradigmi di memoria distribuita o condivisa, o entrambi.
 - 4.
-

First things first

5. Profilare il codice in diverse condizioni (carico di lavoro / dimensione del problema, parallelismo, piattaforme, ...) e **individuare colli di bottiglia e inefficienze.** *→ Analisi su vari casi non solo del programma ma anche del sistema che lo usa*
6. Applicare le tecniche di ottimizzazione modificando i punti critici nel codice per **rimuovere i blocchi di ottimizzazione** e/o per esporre meglio i parallelismi intrinseci istruzione/dati.
7. SE (necessario) VAI al punto 1.

Questo non è un processo semplice e lineare. L'ottimizzazione di un codice può richiedere diversi passaggi di prova ed errore e le architetture moderne sono così complesse e la loro evoluzione così rapida che molto spesso può essere difficile modellare perfettamente le prestazioni di un codice, e anche tecniche promettenti a volte possono fallire.

lascia che il compilatore faccia il suo lavoro

I linguaggi di programmazione sono notazioni per descrivere i calcoli a persone e macchine.

[...] tutto il software in esecuzione su tutti i computer è stato scritto in qualche linguaggio di programmazione.

Ma, prima che un programma possa essere eseguito, deve prima essere tradotto in una forma in cui possa essere eseguito da un computer.

I sistemi software che eseguono questa traduzione sono chiamati **compilatori**.

user → linguaggio → compilatore → linguaggio
macchine → computer

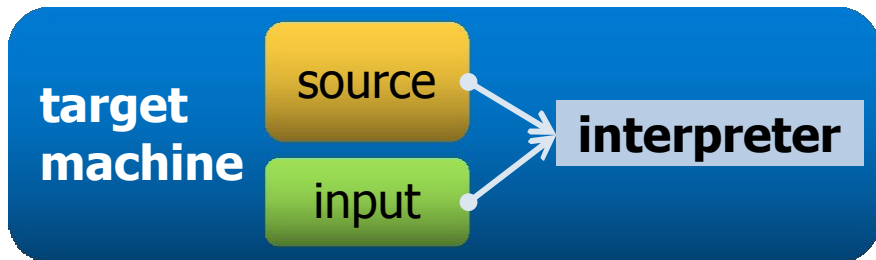
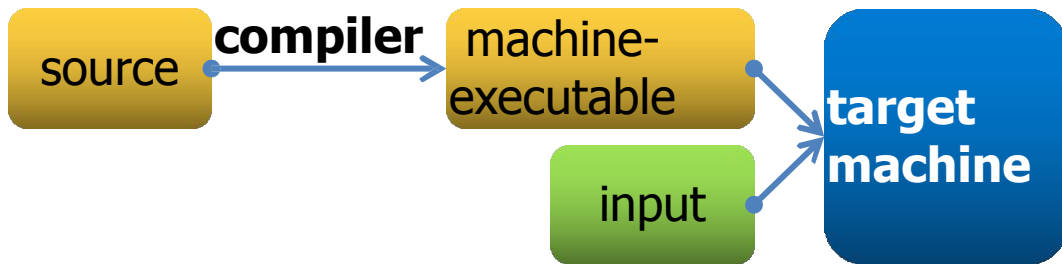
Lascia che il compilatore faccia il suo lavoro

In parole semplici, un compilatore è **un programma che traduce un programma da un linguaggio sorgente in un programma *equivalente* in un linguaggio target**, segnalando anche possibili errori in esso (principalmente errori semantici e un sottoinsieme di altri tipi di errori).

Se il linguaggio target è eseguibile da una macchina, può quindi essere chiamato direttamente dalla macchina per elaborare gli input e produrre gli output.

Lascia che il compilatore faccia il suo lavoro

Un **interprete** è un diverso programma di elaborazione del linguaggio che esegue direttamente un programma in un dato linguaggio sorgente.



Di solito i **linguaggi compilati** vengono eseguiti molto più velocemente, mentre i **linguaggi interpretati** offrono un'analisi degli errori e una portabilità migliorate.

Lascia che il compilatore faccia il suo lavoro

Quasi sempre, un compilatore è solo una fase di una lunga pipeline.

Un **preprocessore** può analizzare il sorgente includendo header, espandendo macro ecc. → *risolve i define e le macro / funzioni*

Un **front-end** specifico per un determinato linguaggio (C, C++, Fortran,...) può tradurre il sorgente in un linguaggio ad alto livello di astrazione. → *analizza la sintassi e fa un'altra traduzione intermedia in assembly*

Un **assemblatore** può effettivamente elaborare il codice *assembly* prodotto dal compilatore e produrre un codice macchina *rilocabile* (o codice oggetto) per ogni unità di compilazione. → *trasforma in bytecode*

Un **linker** risolve gli indirizzi di memoria tra diverse sezioni del codice e potenziali riferimenti a librerie. → *risolve le librerie*

Lascia che il compilatore faccia il suo lavoro

esempio:

```
int main ( void )  
{  
    int a = 1;  
    int b = 2;  
    return a + b;  
}
```

codice sorgente in C

compiler



```
...  
mov     DWORD PTR -8[rbp], 1  
mov     DWORD PTR -4[rbp], 2  
mov     edx, DWORD PTR -8[rbp]  
mov     eax, DWORD PTR -4[rbp]  
add     eax, edx  
ret  
...
```

codice sorgente asm x86_64

Lascia che il compilatore faccia il suo lavoro

```
...  
mov     DWORD PTR -8[rbp], 1  
mov     DWORD PTR -4[rbp], 2  
mov     edx, DWORD PTR -8[rbp]  
mov     eax, DWORD PTR -4[rbp]  
add     eax, edx  
ret  
...
```

x86_64 asm source code

assembler



```
...  
c7 45 f8 01 00 00 00  
c7 45 fc 02 00 00 00  
8b 55 f8  
8b 45 fc  
01 d0  
c3  
...
```

**disassembly of the object
code usando objdump**

Lascia che il compilatore faccia il suo lavoro

Tutti i passaggi precedenti sono riproducibili con:

```
cc -o example example.c
```

dove example.c è:

```
int main( void ) { int a = 1;  
    int b = 2; return a + b; }
```

Prova a dare un'occhiata ai risultati di

```
objdump -d example
```

Lascia che il compilatore faccia il suo lavoro

I compilatori sono anche in grado di eseguire un' **analisi sofisticata** del codice sorgente in modo da produrre un codice target (solitamente un codice assembly) **altamente ottimizzato per una data architettura target.**

Lascia che il compilatore faccia il suo lavoro

Come linea guida generale, tieni presente che "ottimizzazione" significa

"lascia che il compilatore estragga il massimo dal tuo codice"

I compilatori sono davvero bravi e hanno una profonda conoscenza dell'hardware su cui stanno girando.

Quindi, per prima cosa, impara come:

- scrivere codice non offuscato
- progettare un buon layout delle strutture dati
- progettare un "buon" flusso di lavoro
- sfruttare le moderne architetture out-of-order, super-scalari, multi-core

Quale compilatore?

I compilatori offrono molte opzioni, quindi la prima mossa intelligente è leggere il manuale.

Tuttavia, esistono standard in modo da poter ottenere immediatamente risultati di base attesi con ogni compilatore decente.

compilare un codice sorgente

```
cc source_name -o
```

compilare un sorgente con info per debug

```
executable_name  
cc source_name -g -o executable_name
```

compilare un sorgente con ottimizzazioni

```
cc -O $n$  source_name -o executable_name
```

dove n è tipicamente 1, 2, 3

quanto deve essere aggressiva l'ottimizzazione

compilatori C/C++ più utilizzati:

gnu (gcc), clang, pgi, intel

Date un'occhiata al progetto godbolt:

<https://godbolt.org>

Ottimizzazione del compilatore

Livello di Ottimizzazione: On

Non è garantito che -O3, sebbene spesso generi un codice più veloce, sia ciò di cui hai realmente bisogno.

Ad esempio, a volte ottimizzazioni costose possono generare più codice che su alcune architetture (ad esempio con *cache più piccole*) viene eseguito più lentamente, e l'uso di -Os può portare a risultati sorprendenti.

Tieni presente che i compilatori moderni consentono ottimizzazioni specifiche locali o flag di compilazione.

In gcc per esempio:

```
__attribute__ ((__option__ ("...")))  
__attribute__ ((optimize(n)))
```

Compilare per modelli specifici di CPU

Livello di Ottimizzazione: native

Il compilatore conosce l'architettura su cui sta compilando, ovviamente. Tuttavia, genererà un codice portabile, cioè un codice che può essere eseguito su qualsiasi CPU appartenente a quella classe di architettura. Ad esempio: x86_64, x86_32, ARM, POWER9, sono tutte classi di architettura.

Oltre a un set generale di istruzioni che tutte le CPU di una data classe possono comprendere, i modelli specifici hanno ISA specifici diversi che non sono compatibili con altri (normalmente si ha la retrocompatibilità).

Utilizzando l'opzione appropriata (in gcc `-march=native -mtune=native`, in icc `-xHost`), il compilatore ottimizzerà esattamente per la CPU specifica su cui sta girando, molto probabilmente producendo un codice più performante per essa.

Usare profiling automatico

I compilatori (gcc, icc e clang) sono in grado di istruire il codice in modo da generare informazioni in fase di esecuzione da utilizzare in compilazioni successive.

Conoscere i tipici modelli di esecuzione consente al compilatore di eseguire ottimizzazioni più mirate, soprattutto se sono presenti diversi branch.

Per gcc:

```
gcc -fprofile-arcs
```

```
< ... run ... >
```

```
gcc -fbranch-probabilities
```



Specifico per predizione branch

```
gcc -fprofile-generate
```

```
< ... run ... >
```

```
gcc -fprofile-use
```



Più in generale; abilita anche

-fprofile-values

-fpreorder-functions

Ottimizzazione guidata

Ricorda la memoria

Allocazione di memoria

Cerca di allocare **memoria contigua** e di **riutilizzarla in modo efficiente** evitando la frammentazione

↑
Ridurre i tempi di accesso!

Alcuni suggerimenti sul C

Classi di Storage

- `extern`
Variabili globali, che esistono per sempre.
- `auto`
Variabili locali, allocate sullo stack per un ambito limitato e poi distrutte. Devono essere inizializzate.
- `register`
Suggerisce che il compilatore inserisca questa variabile direttamente in un registro della CPU.

Alcuni suggerimenti sul C

Qualificatori di variabile

- `const`
Indica che questa variabile non verrà modificata nell'ambito di pertinenza.
- `volatile`
Indica che questa variabile è accessibile e modificabile dall'esterno del programma.
- `restrict`
Un indirizzo di memoria è accessibile solo tramite il puntatore specifico.

Memory aliasing

Uno dei principali ostacoli all'ottimizzazione, probabilmente il primario, è un uso scorretto dei riferimenti alla memoria.

Considera le due funzioni seguenti:

```
void func1 ( int *a, int *b ) {  
    *a += *b;  
    *a += *b; }
```

```
void func2 ( int *a, int *b ) {  
    *a += 2 * *b; }
```

Memory aliasing

Un'analisi incauta potrebbe concludere che un compilatore, o anche un programmatore, dovrebbe trasformare immediatamente `func1()` in `func2()` perché, avendo tre riferimenti di memoria in meno, dovrebbe produrre un codice assembly migliore.

Tuttavia, è davvero vero che le due funzioni si comportano esattamente allo stesso modo in tutte le condizioni possibili?

Cosa succede se $a = b$, cioè se a e b puntano alla stessa posizione di memoria?

Memory aliasing

a e b puntano alla stessa posizione di memoria, e diciamo che

```
*a = 1: void func1 ( int *a, int *b ) {  
    *a += *b;    -> *a e *b adesso contengono 2  
    *a += *b;    -> *a e *b adesso contengono 4  
}
```

```
void func2 ( int *a, int *b )  
    *a += 2 * *b; -> *a e *b adesso contengono 3  
}
```

Questa condizione, cioè quando 2 variabili puntatore fanno riferimento allo stesso indirizzo di memoria, è chiamata **aliasing della memoria** ed è un importante blocco delle prestazioni in quei linguaggi che consentono l'aritmetica dei puntatori come C e C++.

Memory aliasing

A causa degli alias, il compilatore non può tenere in memoria "a" perché se "b" modifica a perché è lo stesso indirizzo si rischia che a non sia aggiornato.

Il qualificatore *restrict*



es_memory_aliasing.c

```
void func1 ( int *a, int *b ) {  
    *a += *b;  
    *a += *b;  
}
```

Il compilatore non può ottimizzare l'accesso ad a e b perché non può assumere che a e b puntino alle stesse posizioni di memoria o, in generale, che i riferimenti non si sovrappongano mai.

Questo è chiamato *aliasing*, formalmente proibito in FORTRAN: che è il motivo per cui in alcuni casi fortran può compilare in eseguibili più veloci senza che si presti alcuna attenzione. Aiuta il compilatore C a fare il massimo sforzo, scrivendo un codice pulito o usando *restrict* o usando le opzioni *-fstrict-aliasing* *-Wstrict-aliasing*.

Memory aliasing

```
void func1 ( int *restrict a, int  
*restrict b ) {  
    *a += *b;  
    *a += *b;  
}
```

Usa il qualificatore *restrict*

Ora stai dicendo al compilatore che le regioni di memoria a cui fanno riferimento *a* e *b* non si sovrapporranno mai.

Quindi, si sentirà sicuro nell'ottimizzare gli accessi alla memoria il più possibile (evitando sostanzialmente di rileggere le posizioni)

Lascia che il compilatore faccia il suo lavoro

Come linea guida generale, tieni presente che "ottimizzazione" significa

"lascia che il compilatore estragga il massimo dal tuo codice"

I compilatori sono davvero bravi e hanno una profonda conoscenza dell'hardware su cui stanno girando.

Quindi, per prima cosa, impara come:

- scrivere codice non offuscato
- progettare un buon layout delle strutture dati
- progettare un "buon" flusso di lavoro
- sfruttare le moderne architetture out-of-order, super-scalari, multi-core

Scrivere codice non offuscato

- Scrivere codice non offuscato
 - evitare l'aliasing della memoria
 - rendere chiaro a cosa serve una variabile e quando
 - prendersi cura dei cicli
 - mantenere sotto controllo i branch condizionali
 - progettare un buon layout della struttura dati
 - progettare un "buon" flusso di lavoro
 - sfruttare le moderne architetture out-of-order, super-scalar, multi-core
-

Progettare le strutture dati

- scrivere codice non offuscato
- progettare un buon layout della struttura dati
 - essere cache-friendly (ma ignaro)
 - ciò che viene usato insieme, rimane insieme
 - essere NUMA-conscious → ogni processore ha la sua RAM ed è bene progettare il codice per evitare cambi
 - evitare il false-sharing nei core multi-threaded
- progettare un "buon" flusso di lavoro
- sfruttare le moderne architetture out-of-order, super-scalar, multi-core

struct o
array

Progettare il flusso di lavoro

- scrivere codice non offuscato
 - progettare un buon layout della struttura dati
 - Progettare un "buon" flusso di lavoro
 - Il compilatore sarà in grado di ottimizzare i branch e i modelli di accesso alla memoria
 - Il prefetching funzionerà meglio
 - Semplificare l'utilizzo del multi-threading
 - sfruttare le moderne architetture out-of-order, super-scalar, multi-core
-

Sfrutta le moderne architetture

- scrivere codice non offuscato
- progettare un buon layout della struttura dati
- progettare un "buon" flusso di lavoro
- Sfrutta le moderne architetture multi-core, superscalari e out-of-order
 - lascia che il compilatore sfrutti il pipelining attraverso l'ordinamento delle operazioni e l'unloop
 - lascia che il compilatore sfrutti le capacità di vettorizzazione delle CPU
 - pensa in modo basato sui task e sui dati

Overview

Questo video è stato generato con strumenti di Intelligenza Artificiale (<https://notebooklm.google/>) utilizzando contenuti e fonti forniti dall'autore del corso.

Il materiale ha esclusivamente funzione di supporto e sintesi degli argomenti trattati a lezione. Non sostituisce il materiale didattico ufficiale, le slide, i testi di riferimento o lo studio individuale.

Il video deve essere inteso come uno strumento integrativo per il ripasso e il consolidamento dei concetti, e non come fonte esaustiva o alternativa allo studio approfondito.



<https://youtu.be/BOMgKdZYhu8>